

Мета роботи

Ознайомитись з особливостями розроблення вбудовуваного інтерфейсу користувача на основі послідовного порту (UART) для реалізації консольного меню. Набути навичок використання переривань для обробки асинхронного введення даних через консоль. Навчитися взаємодіяти з зовнішніми модулями (наприклад, GSM/GPS) за допомогою AT-команд та реалізувати автоматичне розпізнавання (парсинг) відповідей.

Короткий план виконання роботи

1. Виконано ініціалізацію модуля USART для організації консольного обміну даними через UART.
2. Реалізовано діалогову структуру меню.
3. Модифіковано прийом даних так, щоб команди приймалися через переривання RCIE, буферизувались та розпізнавались лише після надходження символу завершення рядка.
4. Реалізовано передачу аргументу разом з командою.
5. Створено інтерфейс за допомогою якої можливо керувати PWM модулем МК дистанційно за допомогою консолі (з'єднання USART).

Код програми

```

1  #include <pl8f4550.h>
2  #include <delays.h>
3
4  #define K1 0x01
5  #define K2 0x02
6  #define K3 0x04
7  #define PIN_Keys PORTE
8  #define PIN_K1 PORTB
9  #define PIN_K2 PORTB
10 #define PIN_K3 PORTB
11 #define Keypad (K1 | K2 | K3)
12
13
14 unsigned int counter;
15 unsigned int keycnt;
16 struct flagsStruct
17 {
18     unsigned int clicking;
19     unsigned int pushing;
20 };
21
22 struct flagsStruct flags;
23 unsigned char press_code;
24 unsigned int adc_value;
25 char buffer[6];
26 char command[100];
27 char commandCharCounter;
28 const char cmd_pwmmon[] = "pwmmon";
29 const char cmd_pwmoff[] = "pwmoff";
30 const char cmd_pwmset[] = "pwmset";
31 const char cmd_pwmmon_response[] = "PWM is on\r";
32 const char cmd_pwmoff_response[] = "PWM is off\r";
33 const char cmd_pwmset_response[] = "PWM is set\r";
34 unsigned char k;
35
36 void systemInit(void);
37 //void TMROInit(void);
38 void USARTInit(void);
39 //void ADCInit(void);
40 void PWMInit(void);
41
42 //void ADCInit(void);
43 void PWMInit(void);
44
45 //void TMROInterrupt(void);
46
47 void sendCharUSART(char);
48 void sendWordUSART(char*);
49 void numberToWord(unsigned int, char*);
50
51 char readCharUSART(void);
52 void operateCommand(char*);
53 char compareWithWord(char*, const char*);
54
55 //unsigned char keyboard(void);
56 void PWMOn(void);
57 void PWMOff(void);
58 void PWMSetup(unsigned char);
59
60 extern void _startup (void);
61 void ISRHigh(void);
62
63 #pragma code main=0x102A
64 void main(void)
65 {
66     TRISB = 0;
67     ADCON1 = 0x0F;
68     counter = 1;
69     keycnt = 0;
70     adc_value = 0;
71
72     systemInit();
73     // TMROInit();
74     USARTInit();
75     // ADCInit();
76     PWMInit();
77
78     while (1)
79     {
80         // sendCharUSART('\f');
81         // if (press_code & 128) sendCharUSART('L');
82         // if (press_code)
83
84         unsigned char multiplier;
85
86         sendCharUSART('B');
87         sendCharUSART((press_code & 7) | '0');
88         multiplier = 1 << ((press_code & 7) - 1);
89         multiplier *= (press_code & 128) ? (8) : 1;
90         sendCharUSART('M');
91         numberToWord(multiplier, buffer);
92         sendWordUSART(buffer);
93         PWMSetup(multiplier);
94
95         for (k = 0; k < 1; k++)
96         {
97             Delay10KTCYx(250);
98         }
99
100 #pragma code interrupt_high_vector=0x1008
101 void interrupt_high_vector(void)
102 {
103     _asm goto ISRHigh _endasm
104 }
105
106 #pragma code
107
108 #pragma interrupt ISRHigh
109 void ISRHigh(void)
110 {
111     // if (INTCONbits.TMROIF)
112     // {
113     //     INTCONbits.TMROIF = 0;
114     //     TMROInterrupt();
115     // }
116     if (PIR1bits.RCIF)
117     {
118         char receivedChar = readCharUSART();
119
120         char receivedChar = readCharUSART();
121
122         PIR1bits.RCIF = 0;
123         if (receivedChar == '\r')
124         {
125             command[commandCharCounter] = '\0';
126             operateCommand(command);
127             commandCharCounter = 0;
128         }
129         else
130         {
131             command[commandCharCounter++] = receivedChar;
132         }
133         Delay10KTCYx(250);
134     }
135
136 void systemInit(void)
137 {
138     INTCONbits.GIE = 1;
139     INTCONbits.PEIE = 1;
140 }
141
142 //void TMROInit(void)
143 //{
144 //    TOCON = 0b10010001;
145 //    INTCONbits.TMROIE = 1;
146 //    INTCON2bits.TMROIP = 1;
147 //}
148
149 void USARTInit(void)
150 {
151     SPBRGH = 0;
152     BAUDCONbits.BRG16 = 0;
153
154     SPBRG = 77;
155     TXSTAbits.BRGH = 0;
156     TXSTAbits.SYNC = 0;

```

```

154     SPBRG = 77;
155     TXSTABits.BRGH = 0;
156     TXSTABits.SYNC = 0;
157     RCSTABits.SPEN = 1;
158     RCSTABits.CREN = 1;
159
160     TXSTABits.TX9 = 0;
161     TXSTABits.TXEN = 1;
162
163     PIR1bits.RCIE = 1;
164     IPR1bits.RCIP = 1;
165 }
166
167 //void ADCInit(void)
168 //{
169 //    ADCON0 = 0b00000001;
170 //    ADCON1 = 0b00001110;
171 //    ADCON2 = 0b00101110;
172 //    //
173 //    ADCON0bits.GO = 1;
174 //}
175
176 void PWMInit(void)
177 {
178     CCP1CON = 0b00001100;
179     T2CON = 0b00000000;
180     TRISCBits.RC1 = 0;
181     TRISCBits.RC2 = 0;
182     TRISBbits.RB3 = 0;
183 }
184
185 void PWMOn(void)
186 {
187     T2CONbits.TMR2ON = 1;
188     // numberToWord(PR2, buffer);
189     // sendWordUSART(buffer);
190     if (PR2 <= 1 || PR2 >= 240)
191     {
192         // sendCharUSART('o');

```

```

228 //         sendCharUSART('!');
229 //         numberToWord(PR2, buffer);
230 //         sendWordUSART(buffer);
231     }
232     else if (multiplier >= 64 && multiplier < 256)
233     {
234         T2CONbits.T2CKPS = 0;
235         PR2 = ((char)(240 * 16 / multiplier)) - 1;
236     }
237     CCP1L = (char)((PR2 + 1) / 2);
238     // numberToWord(CCP1L, buffer);
239     // sendWordUSART(buffer);
240     T2CONbits.TMR2ON = pwm_state;
241 }
242 else
243 {
244     T2CONbits.TMR2ON = 0;
245 }
246 }
247
248 void sendCharUSART(char data)
249 {
250     while(TXSTABits.TXMT == 0);
251     TXREG = data;
252 }
253
254 void sendWordUSART(char* word)
255 {
256     int i = 0;
257     // sendCharUSART('\f');
258     while(word[i] != '\0' && i != 100)
259     {
260         sendCharUSART(word[i]);
261         i++;
262     }
263     // sendCharUSART('\r');
264 }
265
266 void numberToWord(unsigned int number, char* word)
267 {

```

```

191     {
192         // sendCharUSART('o');
193         T2CONbits.T2CKPS = 2;
194         PR2 = 59;
195         CCP1L = (PR2 + 1) / 2;
196     }
197 }
198
199 void PWMOff(void)
200 {
201     T2CONbits.TMR2ON = 0;
202 }
203
204 //void TMR0Interrupt(void)
205 //{
206 //    unsigned char current_press_code = keyboard();
207 //    if (current_press_code) press_code = current_press;
208 //}
209
210 void PWMSetup(unsigned char multiplier)
211 {
212     char pwm_state = T2CONbits.TMR2ON;
213     if (multiplier)
214     {
215         T2CONbits.TMR2ON = 0;
216         if (multiplier < 16)
217         {
218             T2CONbits.T2CKPS = 2;
219             PR2 = ((char)(240 / multiplier)) - 1;
220             // numberToWord(PR2, buffer);
221             // sendWordUSART(buffer);
222         }
223         else if (multiplier >= 16 && multiplier < 64)
224         {
225             T2CONbits.T2CKPS = 1;
226             PR2 = ((char)(240 * 4 / multiplier)) - 1;
227             // sendCharUSART('!');
228             // numberToWord(PR2, buffer);
229             // sendWordUSART(buffer);
230         }

```

```

265 void numberToWord(unsigned int number, char* word)
266 {
267     register unsigned char n;
268     for(n = 0; n <= 4; n++)
269     {
270         word[n] = '0';
271     }
272     for(n = 0; n <= 4; n++)
273     {
274         word[4-n] = (number % 10) + '0';
275         number /= 10;
276         if (!number) break;
277     }
278 }
279
280 int wordToNumber(char* word)
281 {
282     char i = 0;
283     int number = 0;
284     while (word[i] != '\0')
285     {
286         if (word[i] < '0' || word[i] > '9')
287         {
288             number = 0;
289             break;
290         }
291         number *= 10;
292         number += word[i] - '0';
293         i++;
294     }
295     // sendCharUSART('?');
296     // sendCharUSART(number);
297     return number;
298 }
299
300 char readCharUSART(void)
301 {
302     char receivedByte;
303     receivedByte = RCREG;
304     sendCharUSART(receivedByte);
305 }

```

```

202 {
203     char receivedByte;
204     receivedByte = RCREG;
205     sendCharUSART(receivedByte);
206     return receivedByte;
207 }
208
209 void operateCommand(char* command)
210 {
211     char i = 0;
212     char j = 0;
213     char k = 0;
214
215     char command_formatted[50];
216     char command_body[20];
217     char command_argument[20];
218
219     unsigned long frequency;
220     unsigned char multiplier;
221
222     // while (command[i])
223     // {
224     //     sendCharUSART(command[i]);
225     //     i++;
226     // }
227
228     while (command[i] != '\0')
229     {
230         if (command[i] == '\b' && j > 0)
231         {
232             j--;
233             i++;
234             continue;
235         }
236         command_formatted[j] = command[i];
237         i++;
238         j++;
239     }
240     command_formatted[j] = '\0';
241
242     j++;
243     command_formatted[j] = '\0';
244     i = 0;
245     j = 0;
246     while (command_formatted[i] != ' ' && command_formatted[i] != '\0')
247     {
248         command_body[j] = command_formatted[i];
249         i++;
250         j++;
251     }
252     command_body[j] = '\0';
253     if (command_formatted[i] != '\0') i++;
254     while (command_formatted[i] != '\0')
255     {
256         command_argument[k] = command_formatted[i];
257         i++;
258         k++;
259     }
260     command_argument[k] = '\0';
261     // sendWordUSART(command_body);
262     // sendCharUSART('\r');
263     // sendWordUSART(command_argument);
264     // sendCharUSART('\r');
265
266     if (compareWithWord(command_body, cmd_pwmmon))
267     {
268         PWMOn();
269         sendWordUSART(cmd_pwmmon_response);
270     }
271     else if (compareWithWord(command_body, cmd_pwmoff))
272     {
273         PWMOff();
274         sendWordUSART(cmd_pwmoff_response);
275     }
276     else if (compareWithWord(command_body, cmd_pwmset))
277     {
278         frequency = wordToNumber(command_argument);
279         if (frequency < 4) frequency = 4;
280         if (frequency >= 800) frequency = 799;
281
282         multiplier = frequency / 3.125;
283         if (multiplier < 1) multiplier = 1;
284         if (multiplier > 255) multiplier = 255;
285
286         // sendCharUSART('^');
287         // sendCharUSART(multiplier);
288         // numberToWord(multiplier, buffer);
289         // sendWordUSART(buffer);
290         // sendCharUSART('\r');
291         PWMSetup(multiplier);
292         sendWordUSART(cmd_pwmset_response);
293     }
294 }
295
296 char compareWithWord(char* word1, const char* word2)
297 {
298     int i = 0;
299     while (1)
300     {
301         if (word1[i] == '\0' || word2[i] == '\0') return (word1[i] == '\0') && (word2[i] == '\0');
302         if (word1[i] != word2[i]) return 0;
303         i++;
304     }
305 }

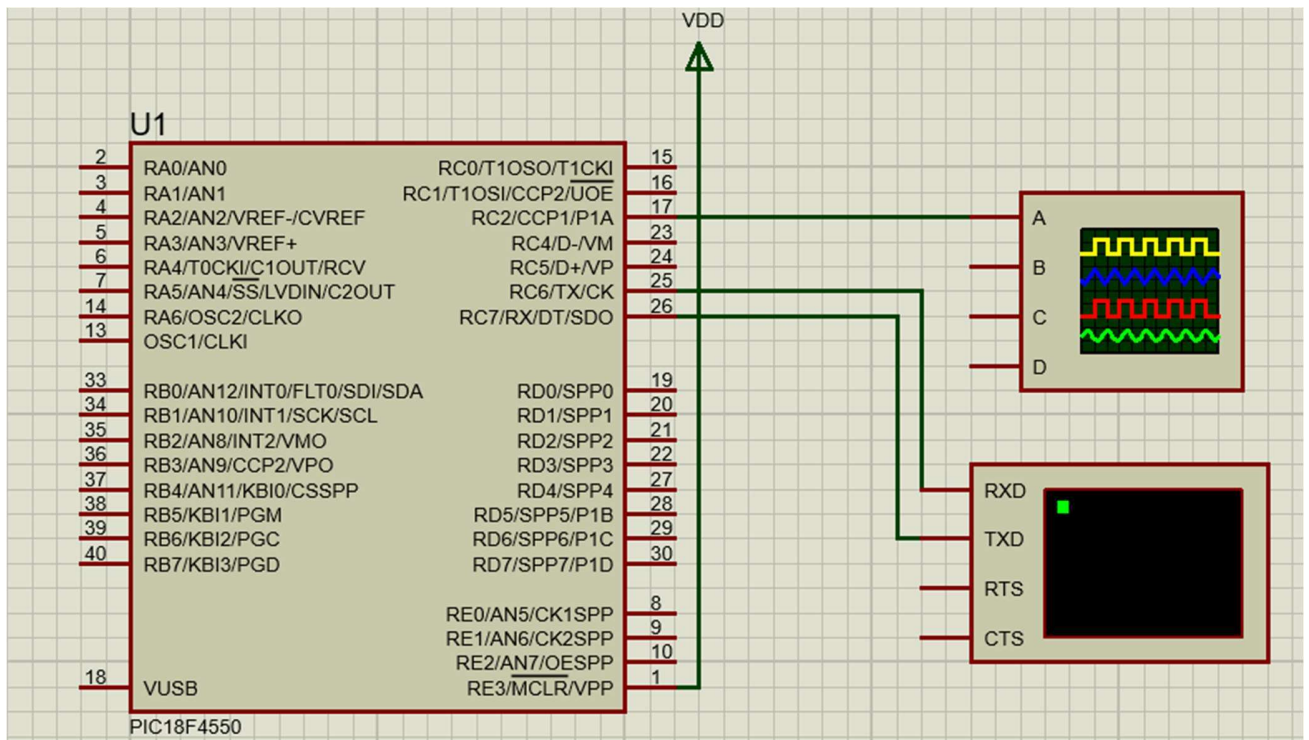
```

```

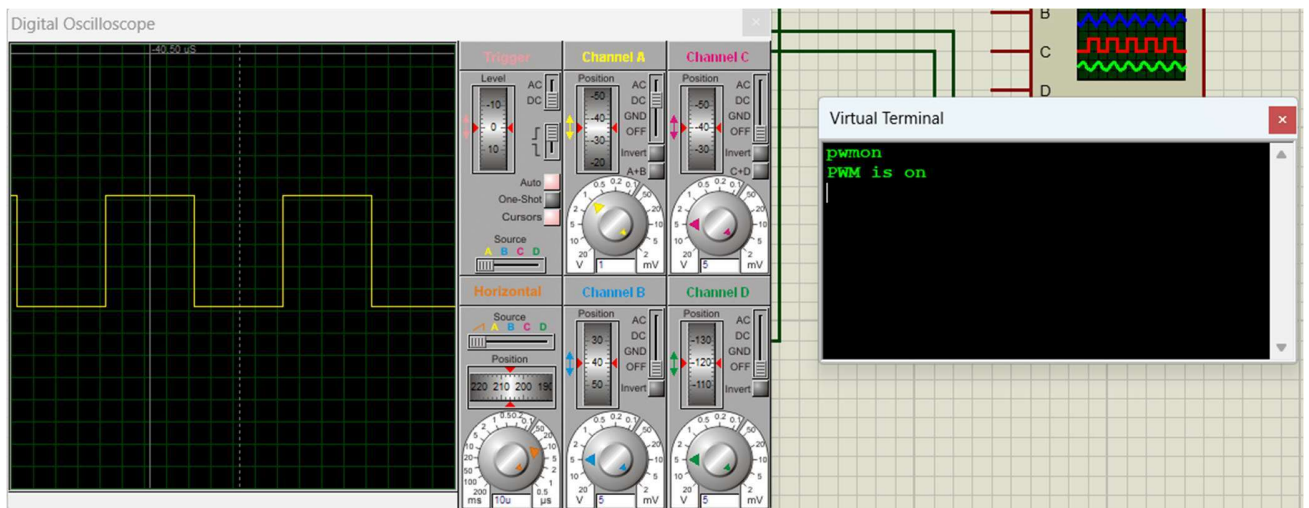
379
380     multiplier = frequency / 3.125;
381     if (multiplier < 1) multiplier = 1;
382     if (multiplier > 255) multiplier = 255;
383     // sendCharUSART('^');
384     // sendCharUSART(multiplier);
385     // numberToWord(multiplier, buffer);
386     // sendWordUSART(buffer);
387     // sendCharUSART('\r');
388     PWMSetup(multiplier);
389     sendWordUSART(cmd_pwmset_response);
390 }
391
392
393 char compareWithWord(char* word1, const char* word2)
394 {
395     int i = 0;
396     while (1)
397     {
398         if (word1[i] == '\0' || word2[i] == '\0') return (word1[i] == '\0') && (word2[i] == '\0');
399         if (word1[i] != word2[i]) return 0;
400         i++;
401     }
402 }

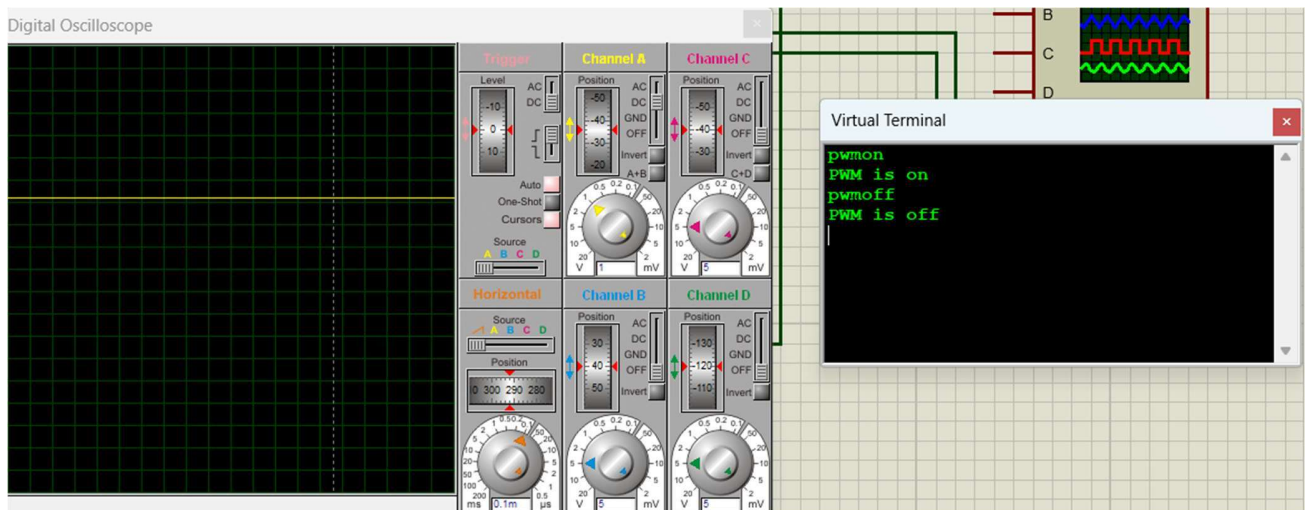
```

Результат виконання у середовищі Proteus (детальніше у демонстрації):

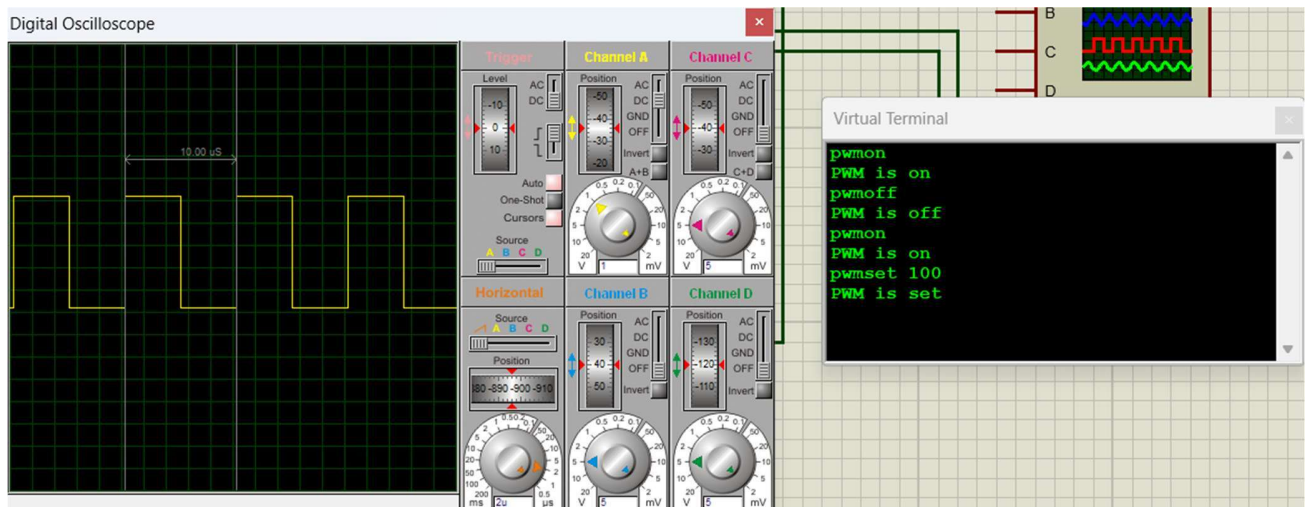


Користувач може вмикати, вимикати та налаштовувати частоту згенерованого ШІМ сигналу.





Приймання даних через UART реалізовано з буферизацією, тому команди обробляються лише після надходження символу завершення рядка. Це дозволяє коректно розпізнавати команди типу `rwmon` та `rwtoff`. Також додано можливість передачі аргументів команди (через символ пробілу):



`rwmsset «frequency»` налаштовує період ШІМ відповідно до заданої частоти, де `frequency` – частота в кГц (роздільна здатність – 3.125 кГц).

У прикладі вище частота = 100 кГц, відповідно період імпульсу = $1 / 100 \text{ кГц} = 10 \text{ мкс}$, що підтверджується на скріншоті.